

What Is DAX?

Data Analysis Expressions (DAX) DAX (Data Analysis Expressions) is a powerful programming language designed for data analysis and calculations. It **uses formulas and functions** to manipulate data and derive meaningful insights. Widely used in major Microsoft products like Power BI, Excel Power Pivot, and SQL Server Analysis Services (SSAS), DAX is used to create dynamic and complex data models and its optimized performance allows it to handle large datasets efficiently.

Importance of DAX

- Custom Calculations
- Advanced Data Analysis
- Interactivity
- Data Relationships
- Performance Optimization

- **1. Measure**

A measure is a dynamic calculation used in data analysis. Measures are calculated on the fly based on the context of the data in your report, such as filters and slicers. It is a single output that is dynamic in nature. Functions like **SUM, MIN, MAX, AVERAGE, COUNT** are used in a Measure.

- **2. Calculated Column**

A calculated column returns a column after an expression or calculation within the columns in data and it is computed row by row when the data is loaded or refreshed. It is stored in the data model and can be used like any other column in your table. **EX = [sales]-[cost]**

- **3. Calculated Table**

A calculated table is a new table created by a DAX formula that returns a table. It can be used to create new tables based on existing data or to aggregate data in a specific way some of the functions that return a data table are **FILTER**, **CALENDAR** and **TOP** etc.

Types of Measures in Power BI

- **Implicit Measures**

Implicit measures are automatically created by Power BI when you drag a numeric field into a visualization. These measures are simple and convenient for quick aggregations but offer limited flexibility.

Example: If you drag the Sales Amount column from the Sales table into a chart, Power BI will automatically create an implicit measure that sums the sales amounts.

- **Explicit Measures**

Explicit measures are user-defined calculations created using DAX (Data Analysis Expressions). These measures provide more control and flexibility, allowing you to define complex calculations tailored to your specific needs.

Example: Total Sales Measure

Total Sales = SUM(Sales[Sales Amount])

- **Key Differences**

Aspect	Implicit Measures	Explicit Measures
Creation	Automatically generated by Power BI	Manually created by users using DAX formulas
Flexibility	Limited to basic aggregations (sum, average, etc.)	Can perform complex calculations and use DAX functions
Reusability	Specific to the visualization were created	Can be reused across multiple visuals and reports
Control	Power BI controls the aggregation logic	Users have complete control over the calculation logic
Example	Dragging Sales Amount into a chart	Total Sales = SUM(Sales[Sales Amount])

Arithmetic Operators

Arithmetic operator	Meaning	Example
+ (plus sign)	Addition	3+3
– (minus sign)	Subtraction or sign	3–1–1
* (asterisk)	Multiplication	3*3
/ (forward slash)	Division	3/3
^ (caret)	Exponentiation	16^4

Comparison operators

Comparison operator	Meaning	Example
=	Equal to	[Region] = "USA"
==	Strict equal to	[Region] == "USA"
>	Greater than	[Sales Date] > "Jan 2009"
<	Less than	[Sales Date] < "Jan 1 2009"
>=	Greater than or equal to	[Amount] >= 20000
<=	Less than or equal to	[Amount] <= 100
<>	Not equal to	[Region] <> "USA"

Basic Aggregation Function

- **SUM**

Adds all the numbers in a column.

Syntax: SUM(<column>)

Example: Total Sales = SUM(Sales[Amount])

- **SUMX**

Returns the sum of an expression evaluated for each row in a table.

Syntax: SUMX(<table>, <expression>)

Example: *Total Sales = SUMX(Sales, Sales[Quantity] * Sales[Price])*

- **AVERAGE**

Returns the average (arithmetic mean) of all the numbers in a column.

Syntax: AVERAGE(<column>)

Example: Average Sales = AVERAGE(Sales[Amount])

- **AVERAGEX**

Calculates the average (arithmetic mean) of a set of expressions evaluated over a table.

Syntax: AVERAGEX(<table>, <expression>)

*Example: Average Sales = AVERAGEX(Sales, Sales[Quantity] * Sales[Price])*

- **MIN**

Returns the smallest numeric value in a column.

Syntax: MIN(<column>)

Example: Min Sales = MIN(Sales[Amount])

- **MINX**

Returns the smallest numeric value in a column or an expression evaluated over a table.

Syntax: MINX(<table>, <expression>)

Example: MINX(Sales, Sales[Amount])

- **MAX**

Returns the largest numeric value in a column.

Syntax: MAX(<column>)

Example: Max Sales = MAX(Sales[Amount])

- **MAXX**

Returns the largest numeric value in a column or an expression evaluated over a table.

Syntax: MAXX(<table>, <expression>)

Example: MAXX(Sales, Sales[Amount])

- **COUNT**

Counts the number of non-blank values in a column.

Syntax: COUNT(<column>)

Example: Count of Sales = COUNT(Sales[Amount])

- **COUNTROWS**

Returns the number of rows in a table.

Syntax: COUNTROWS(<table>)

Example: COUNTROWS(Sales)

- **COUNTBLANK**

Counts the number of blank cells in a column.

Syntax: COUNTBLANK(<column>)

Example: COUNTBLANK(Sales[Amount])

- **DISTINCTCOUNT**

Counts the number of distinct values in a column.

Syntax: DISTINCTCOUNT(<column>)

Example: DISTINCTCOUNT(Sales[OrderID])

TEXT Function in DAX

- **LEN**

Returns the number of characters in a text string.

Syntax: LEN(<text>)

Example: LEN("Power BI") returns 8.

- **LEFT**

Extracts a specified number of characters from the start of a text string.

Syntax: LEFT(<text>, <num_chars>)

Example: LEFT("Power BI", 5) returns "Power".

- **RIGHT**

Extracts a specified number of characters from the end of a text string.

Syntax: RIGHT(<text>, <num_chars>)

Example: RIGHT("Power BI", 2) returns "BI".

- **MID**

Returns a specified number of characters from the middle of a text string, starting at a specified position.

Syntax: MID(<text>, <start_num>, <num_chars>)

Example: MID("Power BI", 7, 2) returns "BI".

- **FIND**

Finds the starting position of one text string within another text string. It is case-sensitive.

Syntax: FIND(<find_text>, <within_text>[, <start_num>])

Example: FIND("B", "Power BI") returns 7.

- **SEARCH**

Similar to FIND, but it is case-insensitive and accent-sensitive.

Syntax: SEARCH(<find_text>, <within_text>[, <start_num>])

Example: SEARCH("b", "Power BI") returns 7.

- **REPLACE**

Replaces part of a text string with a different text string.

Syntax: REPLACE(<old_text>, <start_num>, <num_chars>, <new_text>)

Example: REPLACE("Power BI", 7, 2, "DAX") returns "Power DAX".

- **CONCATENATE**

Joins two text strings into one.

Syntax: CONCATENATE(<text1>, <text2>)

Example: CONCATENATE("Power", " BI") returns "Power BI".

- **UPPER**

Converts a text string to all uppercase letters.

Syntax: UPPER(<text>)

Example: UPPER("Power BI") returns "POWER BI".

- **LOWER**

Converts a text string to all lowercase letters.

Syntax: LOWER(<text>)

Example: LOWER("Power BI") returns "power bi".

Number Functions

- **DIVIDE**

Used to perform division and handle cases where the denominator might be zero, avoiding division errors.

Syntax: DIVIDE(<Numerator>, <Denominator>, [AlternateResult])

Example: DIVIDE(23,0,0) return 0 because 0 cannot be divide by 0.

remainder of 1.

- **ROUND**

Rounds a number to the specified number of digits.

Syntax: ROUND(<number>, <num_digits>)

Example: ROUND(123.456, 2) returns 123.46, rounding to two decimal places.

Date Functions

- **DATE**

Creates a date from individual year, month, and day components.

Syntax: DATE(year, month, day)

Example: DATE(2024, 10, 1) returns October 1, 2024.

- **YEAR**

Extracts the year from a date.

Syntax: YEAR(date)

Example: YEAR("2024-10-01") returns 2024.

- **MONTH**

Extracts the month from a date.

Syntax: MONTH(date)

Example: MONTH("2024-10-01") returns 10.

- **DAY**

Extracts the day from a date.

Syntax: DAY(date)

Example: DAY("2024-10-01") returns 1.

- **WEEKDAY**

Returns the day of the week for a date, with options for different return types.

Syntax: WEEKDAY(date, [return_type])

Example: WEEKDAY("2024-10-01", 2) returns 2 (Tuesday).

- **TODAY**

Returns the current date.

Syntax: TODAY()

Example: If today is October 1, 2024, TODAY() returns "2024-10-01".

- **NOW**

Returns the current date and time.

Syntax: NOW()

Example: If the current date and time is October 1, 2024, 15:31, NOW() returns "2024-10-01 15:31".

- **FORMAT**

used to format a value (usually a number or date) as text, according to a specified format.

Syntax: FORMAT(<value>, <format_string>)

Example: FORMAT(0.75, "0%") // Result: "75%"

- **DATEDIFF**

The DATEDIFF function returns the difference between two dates in terms of the specified unit (day, month, year, etc.).

Syntax: DATEDIFF(<Start Date>, <End Date>, <Interval>)

Advance DAX Function

- **IF**

Returns one value if a condition is true and another value if it's false.

Syntax: IF(condition, true_value, false_value)

Example: IF([Sales] > 1000, "High", "Low")

- **AND**

Returns TRUE if both conditions are true.

Syntax: AND(condition1, condition2)

Example: AND([Sales] > 1000, [Profit] > 500)

- **OR**

Returns TRUE if at least one of the conditions is true.

Syntax: OR(condition1, condition2)

Example: OR([Sales] > 1000, [Profit] > 500)

- **CALCULATE** ->Important

Evaluates an expression in a modified filter context.

Syntax: CALCULATE(expression, filter1, filter2, ...)

Example: CALCULATE(SUM([Sales]), [Region] = "West")

- **VAR**

Variables are used to simplify and optimize formulas You can define a variable in any DAX expression by using **VAR** followed by **RETURN**.

```
Sales With Discount =  
VAR DiscountRate = 0.1  
VAR TotalSales = SUM(Sales[SalesAmount])  
RETURN TotalSales * (1 - DiscountRate)
```

- **CALENDAR**

Returns a table with a single column of dates between the specified start and end dates.

Syntax: CALENDAR(start_date, end_date)

Example: CALENDAR(DATE(2023, 1, 1), DATE(2023, 12, 31))